EMAIL COMPLIANCE AI AGENT

LangChain Usage & LLM Call Reference

A precise, code-level explanation of where LangChain is used, what is sent to Azure OpenAI GPT-4o, and the exact structure of both LLM calls made per email.

2	2	3
LLM calls per email	Source files use LangChain	LangChain imports total

1. Where LangChain Is Used

LangChain is used in **exactly two source files** and for exactly **two purposes**: creating the Azure OpenAI client, and wrapping messages in typed role containers before sending. Nothing else in the application uses LangChain — no chains, no memory, no agents, no tools, no retrieval, no embeddings.

The three imports

Import	From package	Used for
AzureChatOpenAI	`langchain_openai`	Azure OpenAI client object
SystemMessage	`langchain_core.messages`	Typed system-role message
HumanMessage	`langchain_core.messages`	Typed user-role message

File 1 — src/compliance_agent.py

Analyses each email. Uses AzureChatOpenAI to call the model and SystemMessage/HumanMessage to format the two-message payload.

```
from langchain_openai import AzureChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

# Build the client once on startup
self._llm = AzureChatOpenAI(
azure_endpoint = os.environ['AZURE_OPENAI_ENDPOINT'],
api_key = os.environ['AZURE_OPENAI_API_KEY'],
api_version = '2024-02-15-preview',
azure_deployment = 'gpt-4o',
temperature = 0.0, # deterministic output
max_tokens = 1500,
)

# Send a call
messages = [SystemMessage(content=SYSTEM_PROMPT),
HumanMessage(content=filled_email_prompt)]
response = self._llm.invoke(messages)
raw_text = response.content # plain string JSON from the model
```

File 2 — src/guardrails.py (ComplianceVerifier)

Makes a second, lighter LLM call to verify the initial finding. Identical imports, smaller max_tokens.

```
from langchain_openai import AzureChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage

self._llm = AzureChatOpenAI(
azure_endpoint = os.environ['AZURE_OPENAI_ENDPOINT'],
api_key = os.environ['AZURE_OPENAI_API_KEY'],
api_version = '2024-02-15-preview',
azure_deployment = 'gpt-4o',
temperature = 0.0,
max_tokens = 400, # smaller — only a 4-field confirmation needed
)
```

What LangChain does NOT do in this app: no chains, no memory, no agents, no tools, no retrieval, no embeddings, no vector stores. It is used purely as a typed API client. The raw **openai** SDK could replace it with no other changes.

2. LLM Call 1 — Compliance Analysis

One call per email. Made inside **ComplianceAgent.analyse()**. Sends the compliance expert persona and the full email text; receives a 6-field JSON compliance finding.

File	src/compliance_agent.py	
temperature	0.0	Deterministic — same email always gives same result
max_tokens	1500	Enough for detailed JSON with reasoning and evidence
Calls/email	1	One call, then guardrails check, then Call 2

Message 1 — System (SYSTEM_PROMPT constant)

Sets the AI persona as a financial compliance expert, lists all 6 category definitions with IDs, specifies the exact JSON output schema, and states the rules.

SYSTEM

Persona (sent verbatim):

```
You are a financial compliance expert AI agent specialising in
email surveillance for banking and financial institutions.
```

Categories (sent verbatim — 6 definitions):

```
1. market_manipulation - attempts to artificially influence prices or markets
2. bribery - offering or accepting improper payments / quid pro quo
3. secrecy - sharing confidential or non-public information
4. employee_ethics - code-of-conduct violations, conflicts of interest
5. change_in_communication - moving conversation off monitored channels
6. complaints - formal or informal regulatory/client complaints
```

Required JSON output schema (sent verbatim):

```
You MUST respond ONLY with a valid JSON object (no markdown, no preamble):
{
"categories": ["", ...],
"intent": "",
"confidence": ,
"evidence": ["", ...],
"reasoning": "",
"is_compliant":
}
```

Rules (sent verbatim):

```
- If compliant: return categories=[], is_compliant=true, confidence=1.0
- confidence reflects certainty of non-compliance finding
- evidence must be actual text from the email
- reasoning must be at least one clear sentence
- Do not hallucinate; if unsure lower confidence and explain in reasoning
```

Message 2 — Human (ANALYSIS_TEMPLATE filled with email data)

Contains the actual email. Python string formatting fills the five placeholders with values from the email dict produced by email_reader.py.

USER

Template structure (ANALYSIS_TEMPLATE constant):

```
Analyse the following email for compliance violations.

--- EMAIL START ---
From: {from_}
To: {to}
Subject: {subject}
Date: {date}

{body}
--- EMAIL END ---

Return your analysis as a JSON object following the schema provided.
```

Fields injected from the email dict:

{from_}	email['from']	Sender address
{to}	email['to']	Recipient address
{subject}	email['subject']	Subject line
{date}	email['date']	Sent timestamp
{body}	email['body']	Full email body text

Example filled prompt (real test email):

```
Analyse the following email for compliance violations.

--- EMAIL START ---
From: james.hartley@alphahedge.com
To: sarah.mills@bnkinternal.com
Subject: RE: XTEK position - coordinate before open
Date: 2024-03-12 07:42

Sarah, I spoke with Mike at Vertex last night. We're both going heavy
on XTEK at open - if we coordinate our buy orders around 09:31 we can
push the price up 4-5% before the retail crowd notices. Once it spikes,
we unload. Don't put this in the system. Call me on the personal number.
--- EMAIL END ---

Return your analysis as a JSON object following the schema provided.
```

Model Response — parsed by _parse()

The model returns a JSON string. **_parse()** strips any markdown fences (the model sometimes wraps output in triple-backtick blocks), calls json.loads(), and validates all 6 required keys are present.

OUTPUT

Example model response:

```
{
"categories": ["market_manipulation"],
"intent": "Sender proposes coordinated trades to artificially
inflate XTEK stock price before unloading.",
"confidence": 0.96,
"evidence": [
"coordinate our buy orders around 09:31 we can push the price up 4-5%",
"Don't put this in the system"
],
"reasoning": "The email explicitly proposes coordinating simultaneous
buy orders with an external party to artificially move a
stock price — textbook market manipulation.",
"is_compliant": false
}
```

3. LLM Call 2 — Verification

A second, lighter LLM call made inside **ComplianceVerifier.verify()** in guardrails.py. It acts as an independent auditor: given both the original email and the Call 1 finding, it either confirms or corrects the result.

File	src/guardrails.py	class ComplianceVerifier
temperature	0.0	Same deterministic setting
max_tokens	400	Smaller — 4-field JSON confirmation only
Calls/email	1	Runs after Call 1 and after guardrail validation
confirmed=true	Call 1 values pass through unchanged	
confirmed=false	categories + confidence overwritten	is_compliant recalculated

Message 1 — System (_VERIFY_SYSTEM constant)

Shorter persona — sets the model as a compliance audit verifier. Uses a smaller 4-field schema.

SYSTEM

Full _VERIFY_SYSTEM constant (sent verbatim):

```
You are a compliance audit verifier.
Given an email and a preliminary compliance finding, confirm or correct it.
Respond ONLY with valid JSON:
{
"confirmed": ,
"corrected_categories": ["", ...],
"corrected_confidence": ,
"verification_note": ""
}
```

Message 2 — Human (_VERIFY_PROMPT filled)

Sends both the original email and the complete Call 1 finding so the verifier has full context.

USER

Template (_VERIFY_PROMPT constant):

```
Email:
Subject: {subject}
Body: {body}

Preliminary finding:
Categories : {categories}
Intent : {intent}
Confidence : {confidence}
Reasoning : {reasoning}

Confirm or correct this finding.
```

Fields injected — email dict + Call 1 output:

{subject}	email['subject']	From email dict
{body}	email['body']	From email dict
{categories}	finding['categories']	Output of Call 1
{intent}	finding['intent']	Output of Call 1
{confidence}	finding['confidence']	Output of Call 1
{reasoning}	finding['reasoning']	Output of Call 1

Model Response

OUTPUT

Confirmed (Call 1 values unchanged):

```
{
"confirmed": true,
"corrected_categories": ["market_manipulation"],
"corrected_confidence": 0.96,
"verification_note": "Confirmed — coordinated trading scheme explicitly described."
}
```

Corrected (categories and confidence overwritten before scoring):

```
{
"confirmed": false,
"corrected_categories": ["employee_ethics"],
"corrected_confidence": 0.55,
"verification_note": "Downgraded — phrasing indirect; ethics concern but
not clear market misconduct."
}
```

4. Full Pipeline Data Flow Per Email

Six steps per email. Steps 2 and 4 involve an LLM call. Steps 1, 3, 5, 6 are pure Python with no LLM.

1 **Load email from PDF or Excel** — no LLM

`email_reader.py`

Produces a dict: id, source, from, to, subject, date, body

2 **LLM CALL 1 — Analysis** — LLM

`compliance_agent.py`

AzureChatOpenAI.invoke([SystemMessage, HumanMessage])
Returns: categories, intent, confidence, evidence, reasoning, is_compliant

3 **Guardrail validation** — no LLM

`guardrails.py`

Pure Python rules: required fields, types, confidence range,
valid category IDs, evidence count, reasoning length

4 **LLM CALL 2 — Verification** — LLM

`guardrails.py`

AzureChatOpenAI.invoke([SystemMessage, HumanMessage])
Returns: confirmed, corrected_categories, corrected_confidence, note

5 **Priority scoring** — no LLM

`scoring_engine.py`

compliance_matrix.yaml weights + multipliers = 0-100 score
Assigns band: CRITICAL / HIGH / MEDIUM / LOW

6 **Save to result/** — no LLM

`storage.py`

All fields written to result/compliance_.json
Dashboard loads from result/latest.json

Summary: LangChain provides the Azure OpenAI client (**AzureChatOpenAI**) and two typed message wrappers (**SystemMessage**, **HumanMessage**). Every other module — email reading, guardrail validation, scoring, storage, the Flask API — is standard Python with zero LangChain involvement. Each email generates exactly **2 LLM calls**: Call 1 at max_tokens=1500 and Call 2 at max_tokens=400, both at temperature=0.0.